Overview of Statistical Learning and Modeling – 36-600

Lab 9T: Machine Learning + Trees
Due Wednesday, October 29, 11:59pm

Trishla Nair

To answer the questions below, it may help to refer to the class notes and to Sections 8.1 and 8.3.1-8.3.2 of ISLR (1st ed). *Note, however, that we'll use the rpart package to create trees, which ISLR does not use.* So ISLR is best for looking up background details.

Regression Trees

Data, Part I

We'll begin by importing the heart-disease dataset and log-transforming the response variable, `Cost`:

```
df      <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/heart_di
df      <- df[, -10]
w       <- which(df$Cost > 0)
df      <- df[w, ]
df$Cost <- log(df$Cost)
summary(df)
```

```
##       Cost             Age           Gender    Interventions
##  Min.   : 0.470   Min.   :24.00   Female:608   Min.   : 0.000
##  1st Qu.: 5.090   1st Qu.:55.00   Male  :177   1st Qu.: 1.000
##  Median : 6.235   Median :60.00                Median : 3.000
##  Mean   : 6.314   Mean   :58.73                Mean   : 4.722
##  3rd Qu.: 7.582   3rd Qu.:64.00                3rd Qu.: 6.000
##  Max.   :10.872   Max.   :70.00                Max.   :47.000
##      Drugs           ERVisit       Complications     Comorbidities
##  Min.   :0.0000   Min.   : 0.000   Min.   :0.00000   Min.   : 0.000
##  1st Qu.:0.0000   1st Qu.: 2.000   1st Qu.:0.00000   1st Qu.: 0.000
##  Median :0.0000   Median : 3.000   Median :0.00000   Median : 1.000
##  Mean   :0.3389   Mean   : 3.424   Mean   :0.05478   Mean   : 3.781
##  3rd Qu.:0.0000   3rd Qu.: 5.000   3rd Qu.:0.00000   3rd Qu.: 5.000
##  Max.   :2.0000   Max.   :20.000   Max.   :1.00000   Max.   :60.000
##     Duration
##  Min.   :  0.0
##  1st Qu.: 42.0
##  Median :167.0
##  Mean   :164.7
##  3rd Qu.:281.0
##  Max.   :372.0
```

Question 1

Split the data into training and test sets. Call these `df.train` and `df.test`. Reuse the random number seed that you used when splitting these data prior to learning the multiple linear regression model in a previous lab.

```
set.seed(42)
s        <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]
```

Question 2

Learn a regression tree model and report the test-set MSE. How does this MSE compare with what you observed for the linear model? Is it lower? If so, then the (inherently more flexible) nonlinear regression tree-based model is adapting better to the geometry of the data than the (inherently less flexible) linear model, though with the trade-off of severely limited inferential ability. (Though not completely eliminated, as we'll see.)

```
library(rpart)
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.4.3
```

```
rpart.out  <- rpart(Cost ~ ., data = df.train)
regression.out <- predict(rpart.out, newdata = df.test)

actual_values <- df.test$Cost
mse <- mean((actual_values - regression.out)^2)
print(mse)
```

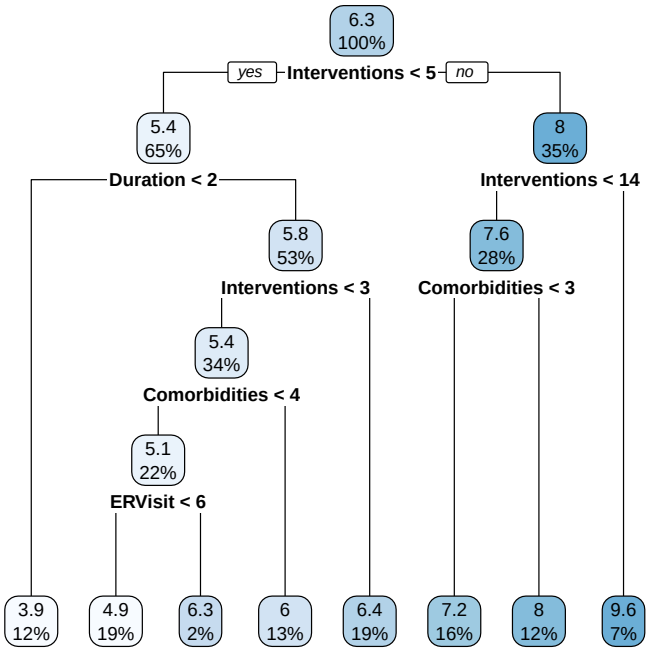
```
## [1] 1.349528
```

```
The MSE is 1.349528. Compared to the linear model, this fith the data better.
```

Question 3

Visualize the tree. Install the package `rpart.plot` and run its namesake function while inputting the results of your tree fit. If you were of a mind to do inference, you'd look for which variables lie near the top of the tree: these are presumably the ones with the most statistical information. (*Note*: because this is a regression tree, the `extra` argument to `rpart.plot()` won't be useful here and you can exclude it from the function call.)

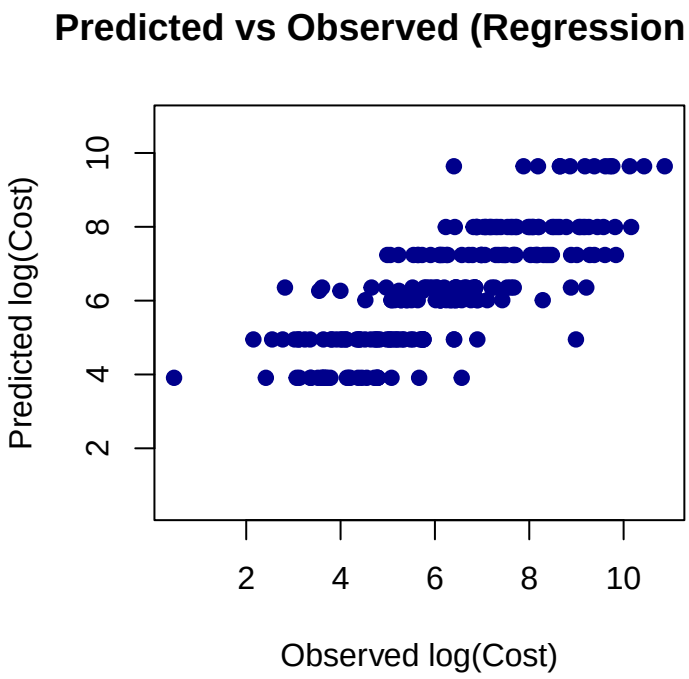
```
rpart.plot(rpart.out)
```



Question 4

Create a diagnostic plot, specifically, the test-set predicted responses (y-axis) versus the test-set observed responses (x-axis). The predictions we're using were those generated in Question 2. For enhanced readability, be sure to set the x limits and the y limits to be the same, and add the $y = x$ line to the plot. Does the plot seem strange to you? If so, and you're not sure what's going on, call us over.

```
plot(actual_values, regression.out,
     xlab = "Observed log(Cost)",
     ylab = "Predicted log(Cost)",
     main = "Predicted vs Observed (Regression Tree)",
     xlim = range(c(actual_values, regression.out)),
     ylim = range(c(actual_values, regression.out)),
     pch = 19, col = "darkblue")
```

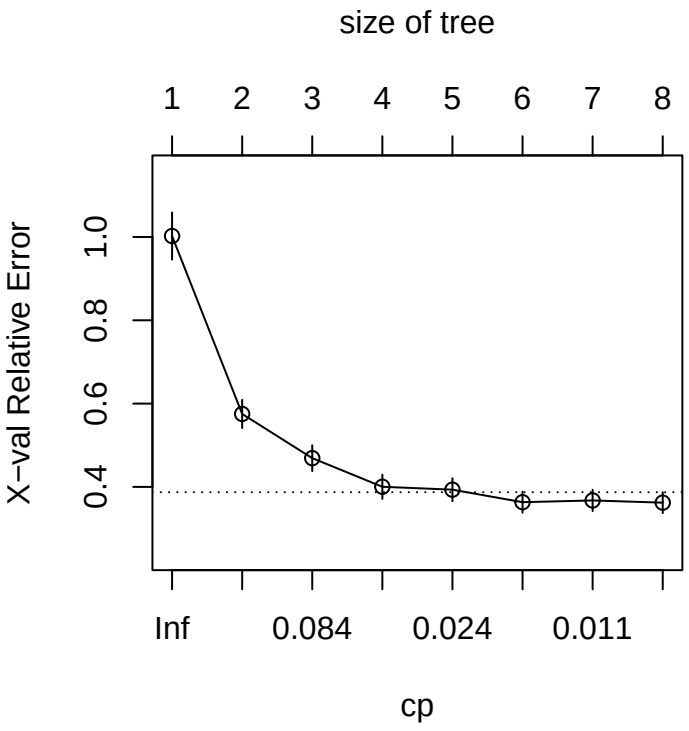


Question 5

Run `plotcp()` with the output of your call to `rplot()` to see if the tree needs pruned. (Yes, it should be "needs to be pruned;" I still flinch at this construction, but you're in Pittsburgh...) As a reminder, you are looking for the leftmost point that lies below the dotted line. If this is not the last point (the point farthest to the right), then `plotcp()` is trying to tell you to prune the tree. Note that depending on how you split the data, you may or may not see evidence of pruning being necessary.

Note that even if pruning is deemed necessary, you do not need to *do* that pruning here. You could, if desired, go back to the code given in today's notes to extract the pruned tree, which you could then use to, e.g., compute an MSE.

```
plotcp(rpart.out)
```



Classification Trees

Now let's turn our attention to classification trees.

Data, Part II

First, let's load in the data on political movements that you looked at in the logistic regression lab:

```
load(url("https://github.com/FSKoerner/F25-36600-data/raw/refs/heads/main/movement.Rdata"))
f <- function(variable, level0 = "NO", level1 = "YES") {
  n              <- length(variable)
  new.variable   <- rep(level0, n)
  w              <- which(variable == 1)
  new.variable[w] <- level1
  return(factor(new.variable))
}
predictors$nonviol      <- f(predictors$nonviol)
predictors$sanctions    <- f(predictors$sanctions)
predictors$aid          <- f(predictors$aid)
predictors$support      <- f(predictors$support)
predictors$viol.repress <- f(predictors$viol.repress)
predictors$defect       <- f(predictors$defect)
levels(response)        <- c("FAILURE", "SUCCESS")
df           <- cbind(predictors, response)
```

```r
names(df)[9] <- "label"
rm(id.half, id,predictors.half, predictors, response)
```

Question 6

Split the data! If you can, match what you did in the logistic regression lab (i.e., with regard to seed-setting).

```r
set.seed(42)
s        <- sample(nrow(df), round(0.7 * nrow(df)))
df.train <- df[s, ]
df.test  <- df[-s, ]
```

Question 7

Your next job is to learn a classification tree. Do that, and output a confusion matrix. (*Note*: the use of the `predict()` function will be a little different here than in the regression setting: use `type = "class"` as an argument, so that the output is not a probability but a classification. You can use this output directly to then construct the confusion matrix.) What is the misclassification rate? If you split your data in the same manner as you did for linear regression, is the MCR lower here?

```r
#library(rpart)
library(rpart.plot)

rpart.out  <- rpart(label ~ ., data = df.train)
class.pred <- predict(rpart.out, newdata = df.test, type = "class")

actual_values <- df.test$label
conf.mat <- table(Predicted = class.pred, Actual = actual_values)
mcr <- 1 - sum(diag(conf.mat)) / sum(conf.mat)
print(paste("Misclassification Rate:", round(mcr, 3)))
```

```
## [1] "Misclassification Rate: 0.308"
```

```
The misclassification rate is 0.308. Compared to the mcr's of the logistic regressio, tis MCR is lower.
```

Question 8

Let's compute the Area Under Curve (AUC) for the decision tree model. Dealing with prediction is again a bit tricky, as the arguments change a bit from model to model, but here, you'll want to run:

```
resp.pred <- predict(rpart.out, newdata = df.test, type = "prob")[, 2]
```

and then mimic material presented in the ROC notes to generate an AUC.

```r
library(pROC)
```

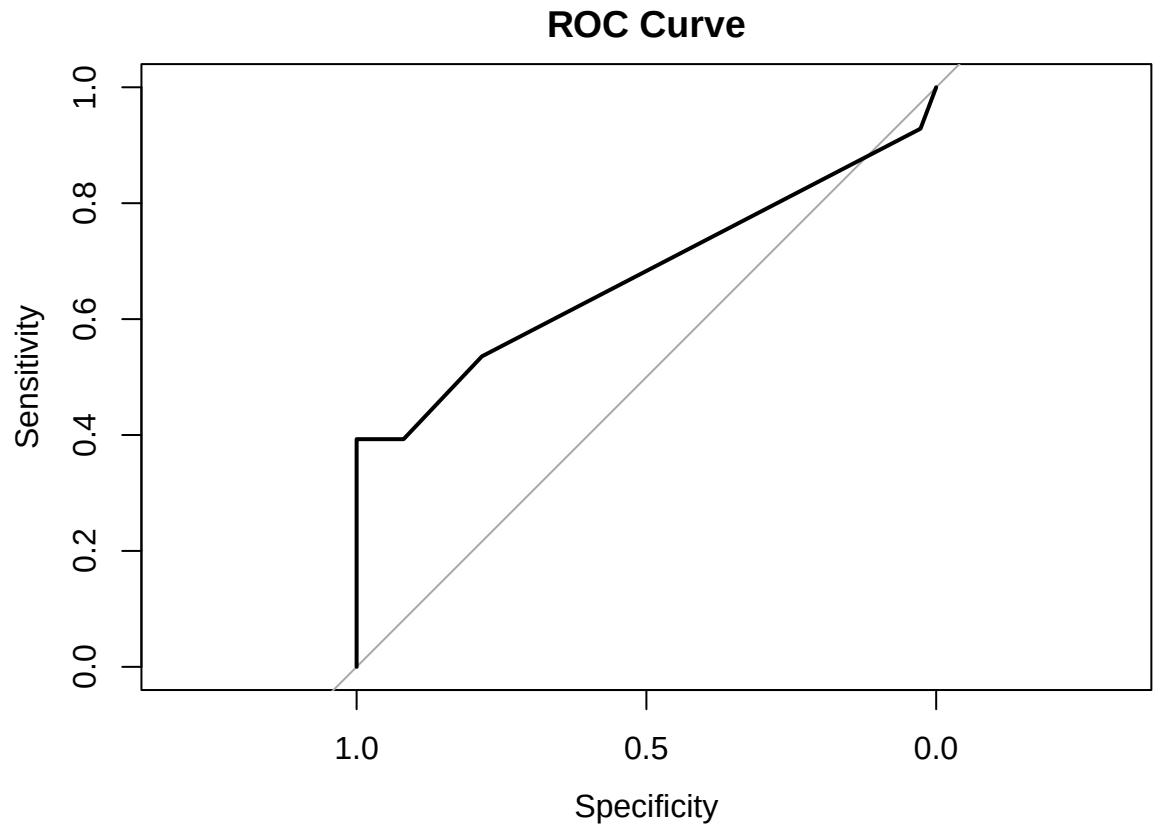
```
## Warning: package 'pROC' was built under R version 4.4.3
```

```
## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
resp.pred <- predict(rpart.out, newdata = df.test, type = "prob")[, 2]
actual_values <- df.test$label

roc.obj <- roc(actual_values, resp.pred, levels = c("FAILURE", "SUCCESS"))
```

```
## Setting direction: controls < cases
```

```
plot(roc.obj, main = "ROC Curve")
```



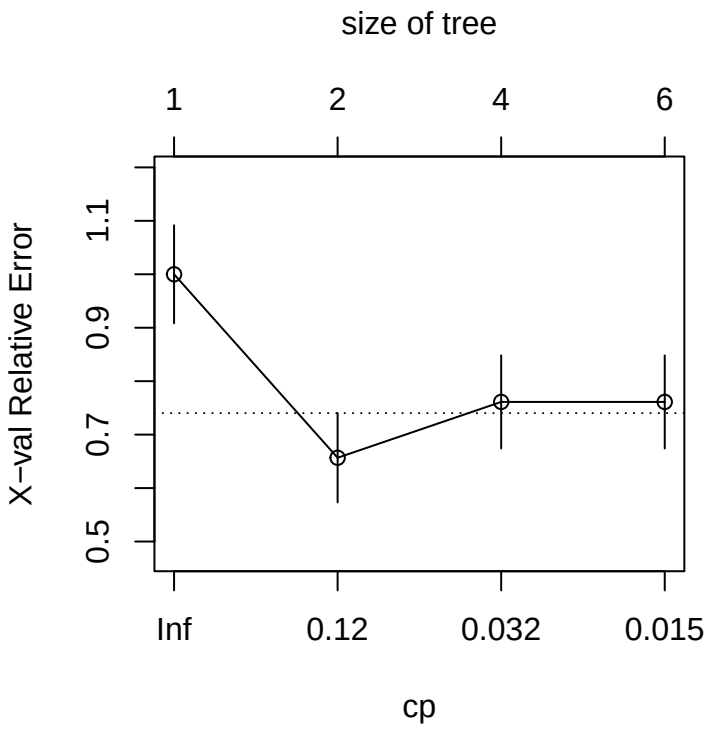

```
auc.value <- auc(roc.obj)
cat("AUC:", round(auc.value, 3))
```

```
## AUC: 0.675
```

Question 9

Plot your classification tree (perhaps with the argument `extra = 104` or `extra = 106`) and determine if it *needs to be pruned* using `plotcp()`. Make a mental note about the pruning.

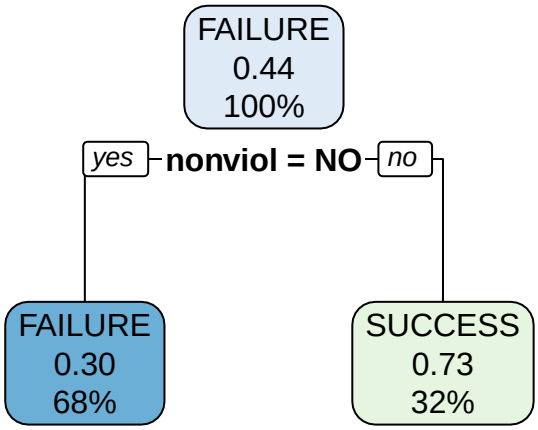
```
plotcp(rpart.out, 106)
```



Question 10

In Question 9, I suspect you saw clear evidence that pruning would be useful. Go ahead, prune the tree and re-plot the pruned tree. Also, compute the misclassification rate: did pruning improve or degrade performance?

```
rpart.pruned <- prune(rpart.out, cp = 0.12)
rpart.plot(rpart.pruned)
```




```
class.prob   <- predict(rpart.pruned, newdata = df.test, type = "prob")[, "SUCCESS"]
actual_values <- df.test$label
conf.mat <- table(Predicted = class.prob, Actual = actual_values)
mcr <- 1 - sum(diag(conf.mat)) / sum(conf.mat)
print(paste("Misclassification Rate:", round(mcr, 3)))
```

```
## [1] "Misclassification Rate: 0.277"
```

The MCR decreased.